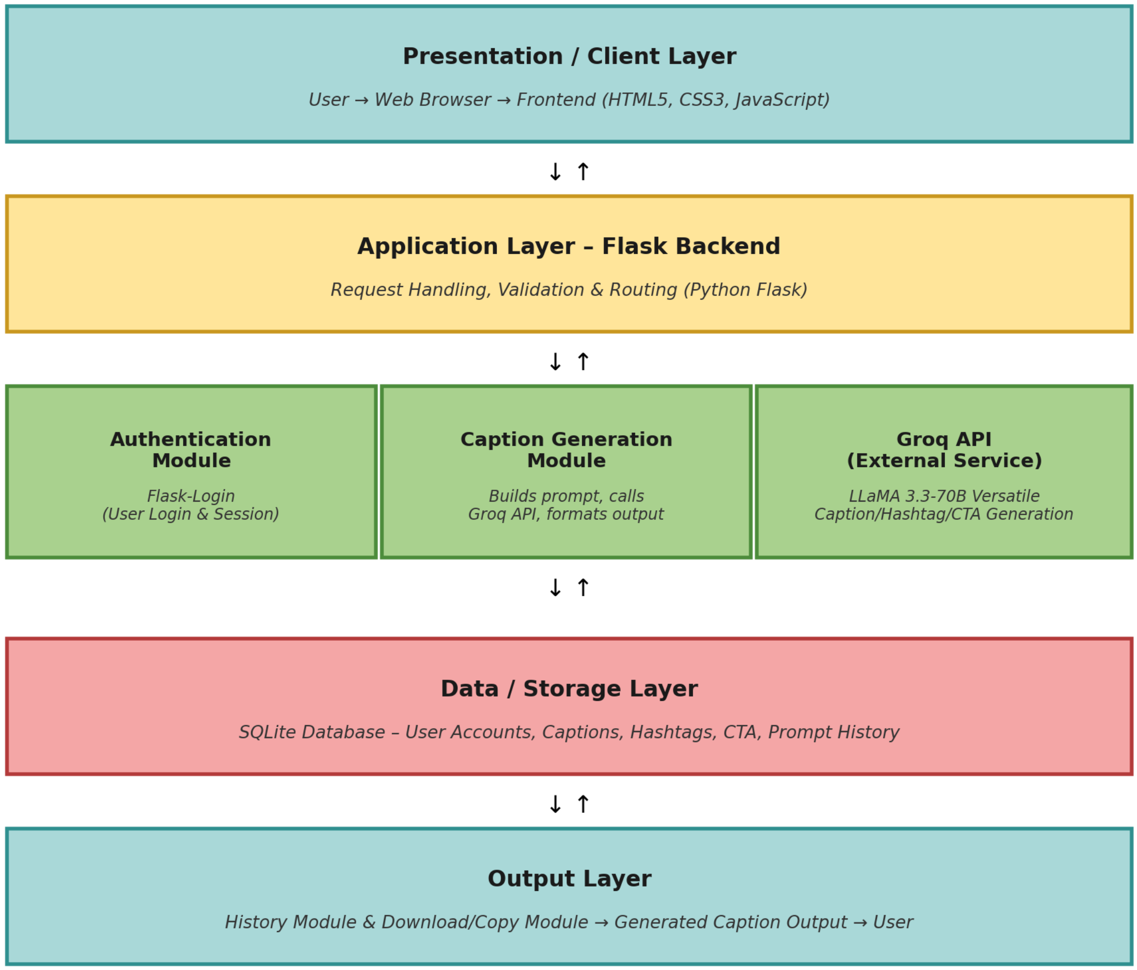
Solution Architecture

Date	07 July 2026
Project Name	CaptionAI: AI Powered Social Media Caption Generator
Maximum Marks	5 Marks

Solution Architecture Overview

CaptionAI follows a client-server architecture where users interact with a web-based interface to generate AI-powered social media captions. The frontend collects user input and sends it to the Flask backend. The backend validates the request, communicates with the Groq API for AI-generated content, stores the generated captions in the SQLite database, and returns the results to the user. The architecture is modular, making the application easy to maintain, extend, and deploy.

Solution Architecture Diagram


Presentation / Client Layer
User → Web Browser → Frontend (HTML5, CSS3, JavaScript)
Application Layer – Flask Backend
Request Handling, Validation & Routing (Python Flask)
Authentication Module
Flask-Login (User Login & Session)
Caption Generation Module
Builds prompt, calls Groq API, formats output
Groq API (External Service)
LLaMA 3.3-70B Versatile Caption/Hashtag/CTA Generation
Data / Storage Layer
SQLite Database – User Accounts, Captions, Hashtags, CTA, Prompt History
Output Layer
History Module & Download/Copy Module → Generated Caption Output → User

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Provides the web-based interface where users register, log in, enter prompts, select the platform and tone, and view, copy, or download the generated captions, hashtags, and CTA suggestions.	HTML5, CSS3, JavaScript
Application Layer (Flask Backend)	Handles all incoming requests from the frontend, validates user input, manages authentication, coordinates communication with the Groq API, and routes data to and from the database. Since CaptionAI is a single lightweight service, Flask itself handles the role of request routing normally played by a separate API gateway.	Python (Flask)
Core Modules (Auth, Caption Generation, History)	The Authentication Module manages user login and sessions. The Caption Generation Module builds the AI prompt and sends it to the Groq API, then formats the returned captions, hashtags, and CTA suggestions. The History Module retrieves and displays previously generated captions, while the Download & Copy Module lets users save or copy results.	Flask-Login, Flask-SQLAlchemy, Requests (Groq API integration)
Database (Data / Storage Layer)	Stores user account details and login credentials, along with generated captions, hashtags, CTA suggestions, and prompt history for future retrieval.	SQLite

External Services

Groq API – The application integrates with the Groq API using the LLaMA 3.3-70B Versatile model. It processes user prompts and generates high-quality captions, hashtags, and call-to-action suggestions with low response time.

Architecture Flow

1. User opens CaptionAI.
2. User logs into the application.
3. User enters a prompt.
4. User selects the social media platform.
5. User selects the preferred tone.
6. The frontend sends the request to the Flask backend.
7. Flask validates the input.
8. Flask sends the request to the Groq API.
9. The AI model generates captions, hashtags, and CTA suggestions.
10. The backend receives the response.
11. Generated content is stored in the SQLite database.
12. The generated results are displayed to the user.
13. The user can copy, download, regenerate, or view previous captions.

Advantages of the Architecture

- Modular and easy to maintain.
- Fast AI response through Groq API.
- Lightweight backend using Flask.
- Efficient local data storage with SQLite.
- Responsive and user-friendly interface.
- Easy integration of additional AI features in future.
- Supports secure user authentication.
- Scalable for future enhancements.

Future Enhancements

- Multilingual caption generation.
- Image-based caption generation.
- AI-powered content scheduling.
- Social media account integration.
- Analytics dashboard for engagement insights.
- Team collaboration features.
- Personalized caption recommendations based on user history.